

MODULE 9 · TROUBLESHOOTING

Probe Problems

The CrashLoop that isn't a crash, and the healthy Pod nobody can reach

A misconfigured probe looks exactly like a real failure

Probes are how Kubernetes decides **restart** vs **remove from Service**.

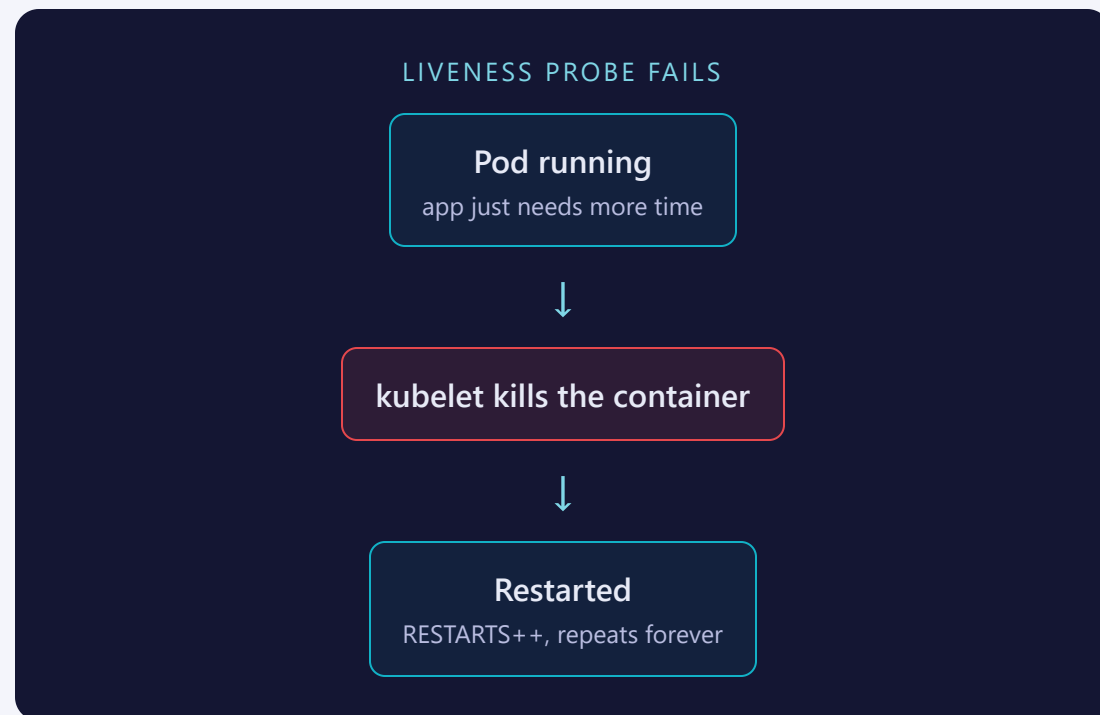
Get the config wrong and it takes the wrong action on a perfectly healthy app — killing it mid-boot, or quietly cutting it off from all traffic.

Both failures **look** like something else: a crash that isn't a crash, an outage with clean logs and a green-looking Pod.

BY THE END YOU CAN

- Spot a CrashLoop that's really a liveness timing problem
- Recognize the 0/1 Ready + zero-endpoints signature
- Tell a wrong-port failure from a wrong-path failure
- Know when a missing startupProbe is the real fix

Two failures, two completely different consequences



What the symptom is telling you

SYMPTOM	PROBE EVENT / PHRASE	ROOT CAUSE
<code>CrashLoopBackOff</code> , RESTARTS climbing, boot interrupted	<code>Liveness probe failed ... connection refused</code> early in lifecycle	Liveness too aggressive for a slow-starting app — add a <code>startupProbe</code>
<code>CrashLoopBackOff</code> , app logs look healthy	<code>connection refused</code> at the probe's port	Probe targets the wrong port
<code>0/1 Ready</code> forever, no restarts, endpoints empty	<code>Readiness probe failed ... statuscode: 404</code>	Readiness path doesn't match a route the app serves
Rollout never reports success	<code>rollout status</code> times out waiting for Ready	Same readiness misconfig — no replica ever becomes Ready

Read the event, then read the probe block

- **describe first** — Events name which probe failed and why: connection refused, statuscode: 404, timeout
- **the probe spec second** — confirms what it's actually checking: port, path, thresholds

ORDER MATTERS

RESTARTS and READY alone don't say *which* probe fired — the Events do.

```
$ kubectl describe pod NAME
# Events: Liveness/Readiness probe failed
#   ...connection refused / statuscode: 404

$ kubectl get pod NAME -o jsonpath=\
  '{.spec.containers[0].livenessProbe}'
$ kubectl get pod NAME -o jsonpath=\
  '{.spec.containers[0].readinessProbe}'
```

The app was fine. The kubelet didn't wait.

App takes ~30s to boot. Liveness has `initialDelaySeconds: 2`, `periodSeconds: 5`, `failureThreshold: 2` — it fails around 12s in, mid-boot. The kubelet kills the container, it restarts, boots again, gets killed again.

Anchor on **RESTARTS climbing**, not STATUS — a fresh restart attempt can briefly read Running even while the loop continues.

FIX

Add a **startupProbe** with a generous budget (e.g. 60s). While it's in flight, liveness & readiness are paused — the app gets time to actually boot.

```
Warning   Unhealthy   Liveness probe failed:
           Get "http://...": dial tcp ...:
           connect: connection refused
Normal    Killing     Container nginx failed
           liveness probe, will be restarted
```

Running, healthy-looking, and invisible to the Service

Pods sit at 0/1 Ready forever with **zero restarts** — nothing is crashing. rollout status never reports success because no replica ever becomes Ready.

The Service has no backends at all. Traffic to it fails with connection refused, even though every Pod is Running and the logs are clean.

THE SILENT-FAILURE MODE

Logs clean, status Running, dashboards green — and nothing routes.

```
$ kubectl get pods -l app=silent-app
NAME                                0/1   Running   0   25s
NAME                                0/1   Running   0   25s

$ kubectl get endpoints silent-app-svc
# legacy API, still populated - hides not-ready IPs
NAME                ENDPOINTS    AGE
silent-app-svc      <none>       30s
```

Same mistake shape, two different fingerprints

	PROBE ON THE WRONG PORT	PROBE ON THE WRONG PATH
Symptom	<code>CrashLoopBackOff</code> , RESTARTS climbing	<code>0/1 Ready</code> forever, RESTARTS stays 0
Event	<code>connect: connection refused</code> — nothing listening there	<code>statusCode: 404</code> — a real HTTP response, wrong one
Which probe	Usually liveness — failure kills the container	Usually readiness — failure just blocks traffic
Fix	Match probe <code>port</code> to the container's actual listening port	Point the probe at a path the app truly serves

Same lesson both times: the probe config and the app's real behaviour drifted apart. Compare them side by side, don't guess.

Where people trip

- **No startupProbe on slow boots.** Liveness kills the app before it finishes starting, even though it would boot fine given time.
- **Anchoring on STATUS, not RESTARTS.** A pod mid-restart can briefly read Running — RESTARTS climbing is the real signal.
- **Assuming CrashLoopBackOff = app bug.** Compare an exit-code crash to a probe-induced kill — very different fix.
- **Liveness as strict as readiness.** A slow dependency blip restarts every pod instead of just pulling it from rotation.

Commands to keep at hand

```
$ kubectl describe pod NAME                # Liveness/Readiness events
$ kubectl get pod NAME                      # RESTARTS + READY columns
$ kubectl get endpoints SVC-NAME            # legacy API - zero endpoints = readiness failing
$ kubectl get pod NAME -o jsonpath='{.spec.containers[0].livenessProbe}'
$ kubectl exec NAME -- curl -s -o /dev/null -w "%{http_code}\n" http://localhost/PATH
```

Tip: the Event tells you which probe fired and why; the probe spec tells you if it's even checking the right thing.

RECAP

Probe problems in one breath

- Liveness fail → kill & restart; readiness fail → pulled from Service, no restart
- CrashLoopBackOff isn't always a crash — it can be liveness killing a slow boot
- 0/1 Ready + <none> endpoints = a readiness probe, not app health
- Missing startupProbe is the #1 fix for slow-starting apps

HANDS-ON

Now run Lab 9.8
[labs/9.8/](#)

Next: [9.9 — Resource & HPA Problems](#)